NAME	SHARMILA S
REGISTER NO	1912260141
DEPARTMENT	B.TECH-ECE
COURSE NAME	PYTHON PROGRAMMING
COURSE CODE	CSA0823
SLOT	C
GITHUB LINK	https://github.com/ssharmila8125-netizen/sharmisCSA08-1912260141

Q1 – Lists and Slicing

1. Python program

2. Moving average pseudocode

START

FOR each index i from 0 to n-7

  Calculate sum of readings[i:i+7]

  Divide by 7

  Store the average

END FOR

STOP

3. Slicing explanation

- readings[7:14] – elements from index 7 to 13.
- readings[-7:] – last 7 readings.
- readings[::7] – every 7th element.

4. List operations and complexity

Operation	**Complexity**

append()	O(1) average
insert()	O(n)
pop() at end	O(1)
sort()	O(n log n)

Why list? Lists store ordered, mutable rainfall records and support slicing.

5. Addition count

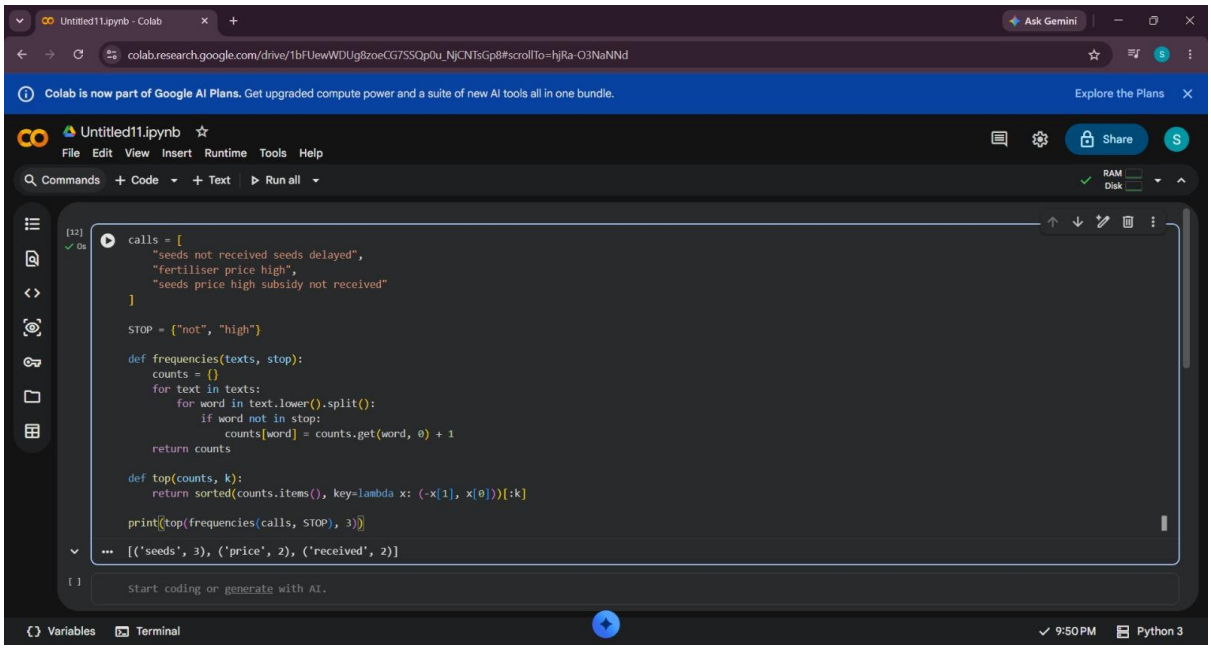
- Naive moving average: \(7(n-6)\) additions = O(7n) = O(n).
- Running sum: approximately \(n-1\) additions = O(n).

6. Late reading insertion

list.insert() takes O(n) because elements may need shifting. A list is still suitable for 365 records, but a date-keyed dictionary may be better for frequent date-based insertion.

7. Alternative: deque

A deque(maxlen=7) maintains a rolling window efficiently. A running sum gives O(n) total processing time.

- List storage: O(n)
- Deque window: O(7) = O(1) additional space
- 

Q2 – Tuples, Mutability and Copying

1. Complete the code

2. Why tuples can be dictionary keys

Tuples are immutable and hashable when their elements are hashable. Lists are mutable and cannot be dictionary keys.

Using a list as a key causes:

TypeError: unhashable type: 'list'

3. Memory diagram

Incorrect grid:

bad —▶ Row 1

├—▶ Row 1

└—▶ Row 1

All rows refer to the same list.

Correct grid:

good —▶ Row 1

├–▶ Row 2

└▶ Row 3

Each row is independent.

4. Copying

- Aliasing: Two variables refer to the same object.
- Shallow copy: Copies the outer object, but nested objects remain shared.
- Deep copy: Copies the outer object and nested objects.

import

A shallow copy is sufficient for simple lists. Deep copy is required when independent nested objects are needed.

5. Why function changes affect caller's list

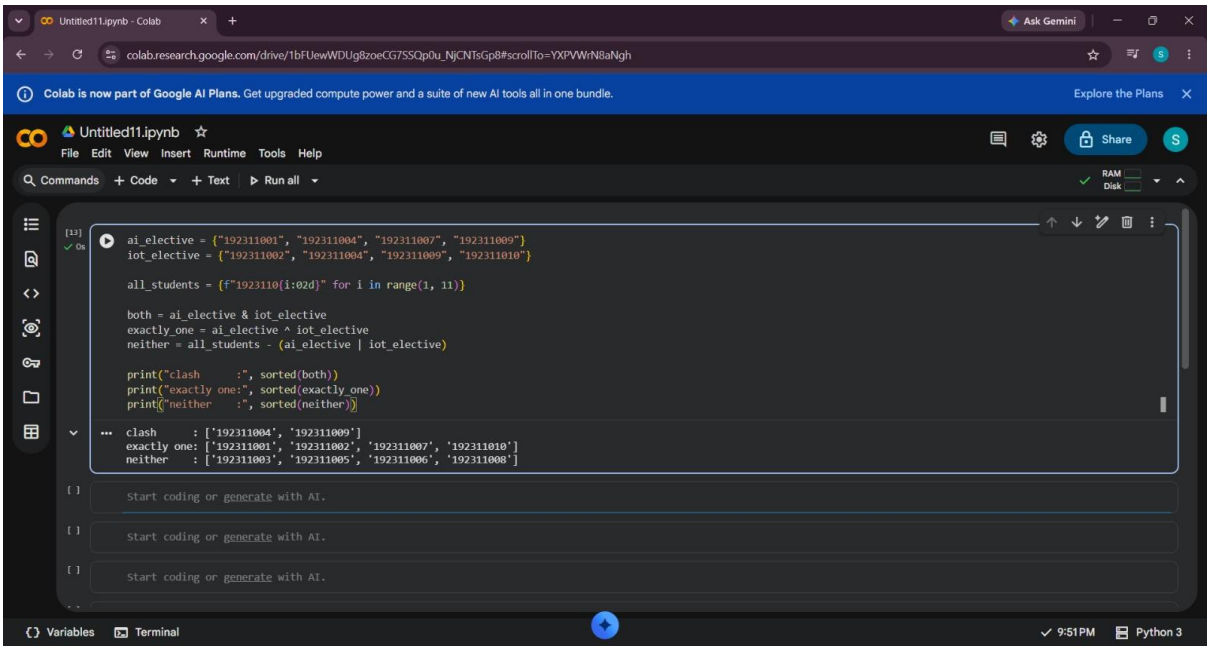
Lists are mutable objects. A function receives a reference to the same list.

The tuple is immutable, but its inner list is mutable.

7. Conclusion

Tuples are suitable for fixed data such as GPS coordinates. Immutability protects data from accidental modification.

8. Alternative: Sparse dictionary grid

- List-of-lists space: O(rows × cols)
- Sparse dictionary space: O(number of occupied zones)
- Average access time: O(1) for both
- 

Q3 – Dictionaries: Farmer Helpline Complaint Analysis

1. Complete the code

2. Explanation

price comes before received because both have count 2, and alphabetical order is used for ties.

3. Dictionary operations

- A dictionary uses hashing to find keys.
- Average lookup complexity: O(1).
- keys() returns keys.
- values() returns values.
- items() returns key-value pairs.

- Python 3.7+ dictionaries preserve insertion order.

4. Complexity analysis

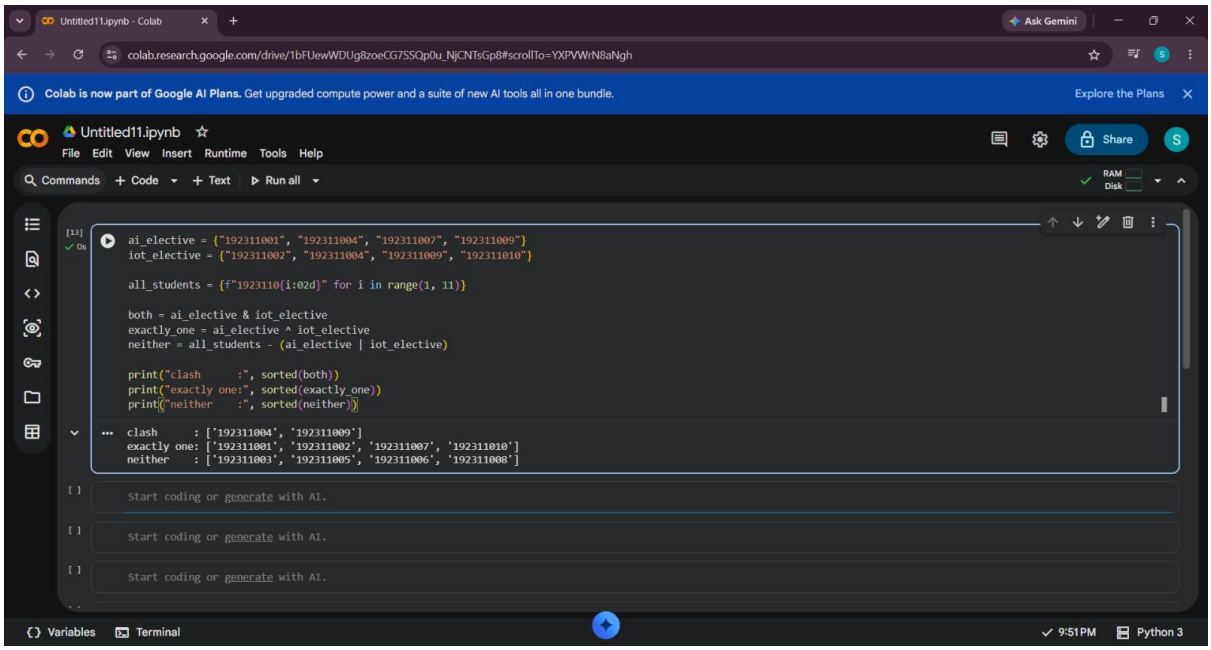
Method	Time complexity
Dictionary counting	O(n) average
list.count() for each distinct word	O(n × d)
collections.Counter	O(n) average

Here, n = total words and d = distinct words.

5. Alternative using Counter

6. Engineering decision

For 100,000 complaints per day, I would use a dictionary or Counter because they provide efficient word counting with O(n) average time.



Q4 – Sets: Elective Registration Clash Detection

1. Complete the code

2. Set operations

Operation	Meaning
A	B	Union
A & B	Intersection

| A - B | Difference |
| A ^ B | Symmetric difference |

3. Rewrite using set methods

4. Explanation

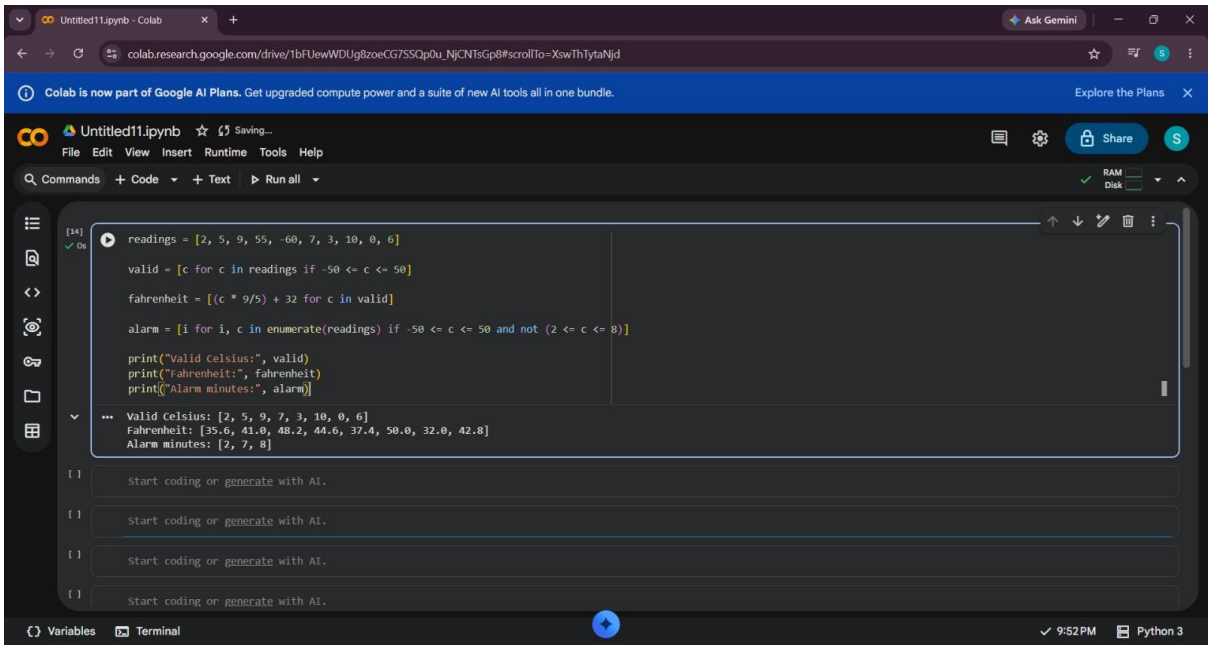
- Sets automatically remove duplicate elements.
- Membership testing in a set is O(1) average.
- Membership testing in a list is O(n).
- Lists cannot be set elements because they are mutable.
- Tuples can be set elements if all their elements are hashable.

5. Complexity analysis

Method	Time
Nested loops	O(n × m)
Sorting and merging	O(n log n + m log m)
Set operations	O(n + m) average

6. Conclusion

Lists should be converted into sets when duplicate removal and fast membership testing are needed. Converting to a set loses duplicate information and original ordering.



Q5 – List Comprehension: Cleaning Sensor Readings

1. Python program

2. General form

3. Map and filter

4. Dictionary and set comprehension

5. List comprehension vs generator

- List comprehension creates a complete list in memory.
- Generator expression produces values one at a time.

6. Complexity

- Time: O(n) for all three approaches.
- List comprehension space: O(n).
- Generator expression extra space: O(1).

A comprehension is best for simple transformations. A normal loop is better for complex logic.

Q6 – Nested Data: Railway Timetable (7 Marks)

1. Python program

2. Data structure explanation

- Dictionary: Stores train details using train number as key.
- List: Stores stations in travel order.
- Tuple: Stores fixed station, arrival and departure information.
- Set: Stores running days without duplicates.

3. Accessing departure time

This accesses the departure time of the third station.

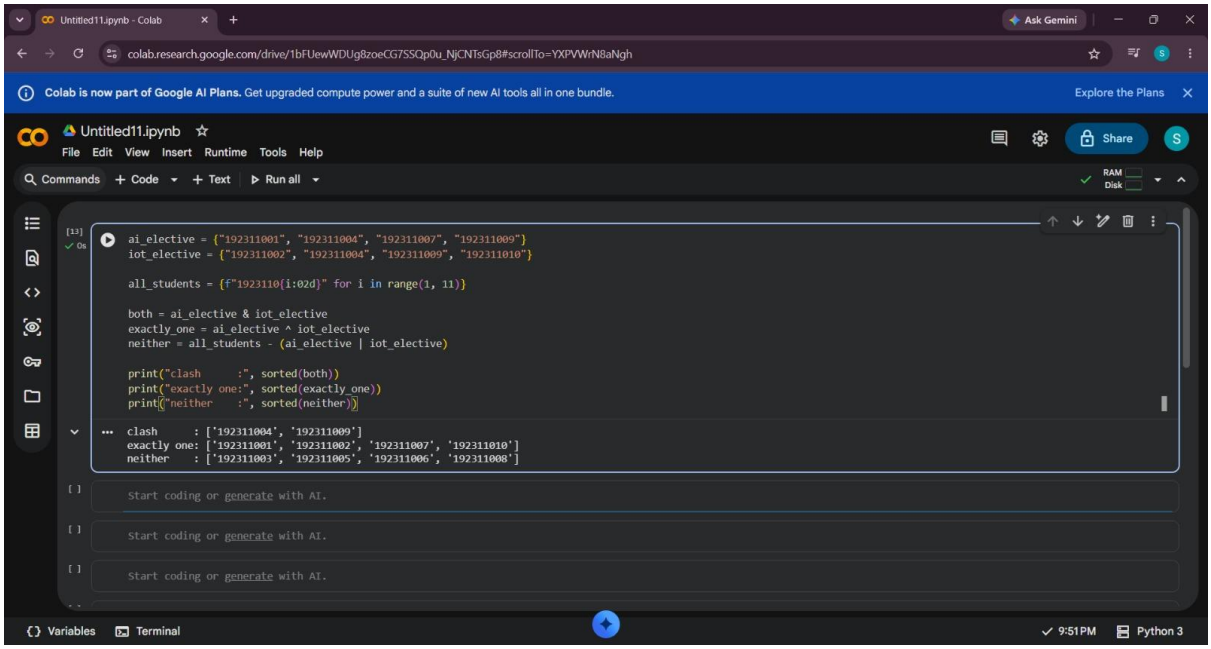
4. Complexity analysis

Let T be the number of trains and S the average number of stops.

- Searching without an index: O(T × S).
- Searching with station index: O(k), where k is the number of trains at that station.
- Index space: O(total stops).

5. Alternative: Station index

Conclusion: An index improves query speed but requires additional memory and updating whenever timetable data changes.



Q7 – Design a Contact-Merging Program from First Principles (10 Marks)

Stage 1 – Interpret the problem (2 Marks)

Input: Contacts from multiple phones and SIM cards.

Output: A merged contact list with unique people and their phone numbers and email addresses.

Rules:

- Phone numbers should be normalized.
- Contacts with the same normalized phone number are considered the same person.
- Names and email addresses should be stored without duplication.

Data structures:

- Dictionary: Stores contacts using normalized phone numbers.
- Set: Stores unique phone numbers and email addresses.
- **Stage 2 – Design (3 Marks)**
- Number normalization:
- Remove spaces, hyphens and parentheses.

- Remove +91 country code for Indian numbers.
- Store the normalized number.
- Pseudocode:

START

Create an empty dictionary

FOR each contact:

  Normalize phone number

  IF normalized number exists:

    Add name, phone and email to existing contact

  ELSE:

    Create a new contact

Remove duplicate numbers and emails

Print merged contacts

STOP

Stage 3 – Implementation Expected result

The first two contacts are merged because their normalized phone numbers are identical.

The third contact remains separate.

Stage 4 – Defend the design (2 Marks)

Why dictionary?

A dictionary provides O(1) average lookup, while comparing every contact with every other contact takes $O(n^2)$.

How are different people with the same name handled?

They are stored separately if their normalized phone numbers differ.

One limitation:

Two people sharing the same phone number may be incorrectly merged.

Complexity analysis

Method	Time complexity	Space
Dictionary-based merging	O(n) average	O(n)
Pairwise comparison	$O(n^2)$	O(n)

For 1,000 contacts, dictionary-based merging is faster than pairwise comparison.
For 100,000 contacts, the difference becomes much more significant.